

iOS Autoplay Restriction — Impact Analysis & Fix for ARRISE

The Problem

Apple's WebKit policy blocks `<video>.play()` on **any video with an audio track** unless it's triggered inside a **user gesture event handler** (touchend, click, etc.). A `<video>` element *can* autoplay if it has the `muted` and `playsinline` attributes, but the moment audio is involved, iOS kills it silently — no error, no exception, just a rejected Promise.

Where This Hits Your Codebase

1. `CDNARVideoController.cs` — The Silent Killer

Your persistent VideoPlayer trick is brilliant for **Android** and **Desktop**, but on iOS Safari it has two fatal autoplay violations:

```
// Awake() – Line 82: Fires with ZERO user interaction
videoPlayer.Prepare();

// OnEnable() – Line 164: Called by AR tracker, not by a tap
videoPlayer.Play();

// OnPrepareCompleted() – Line 172: Callback, not a gesture
videoPlayer.Play();
```

Unity WebGL's `VideoPlayer` is a thin wrapper around a hidden `<video>` HTML element. When your C# calls `.Play()`, Unity's JavaScript bridge calls `videoElement.play()` under the hood. On iOS, **this returns a rejected Promise** because it wasn't triggered by a user tap — but Unity swallows the rejection silently.

Result: Video appears to load forever. The loading spinner never goes away. The user sees nothing.

2. WebGL Templates — The Webcam is Fine, The Video is Not

Your webcam `<video>` element is already correctly set up:

```
<!-- ☒ This works because it's muted + playsinline -->
<video id="webcam-video" muted autoplay playsinline ...></video>
```

The **webcam feed** autoplays fine because it's `muted`. The AR **content videos** (like the CultGym promo, the Wedding Invitation video, etc.) are the ones that break because they have audio tracks in the `.mp4` file.

3. The Auto-Start Flow Skips User Gesture

Both templates call `StartAR()` automatically:

```
// iTracker6/index.html – Line 253
StartAR(); // Automatically called instead of waiting for user click
```

This means Unity boots without any user tap. By the time the AR tracker finds an image and tries to play a video, there has been **no tap event in the browser's gesture chain**, so iOS blocks `.play()`.

The Two Fixes

Fix A: The "Tap-to-Start" Gate (Recommended ☒)

[!IMPORTANT] This is the industry-standard approach used by MindAR, 8thWall, and every successful WebAR platform.

The idea: Force a single user tap *before* the AR experience loads. This tap "unlocks" the browser's media playback policy for the entire session. It also doubles as a polished onboarding moment.

Step 1: Add a Tap Gate to the WebGL Template

Replace the auto-`StartAR()` call with a user-initiated flow:

```
-          await LoadWebcams();
-          StartAR(); // Automatically called instead of waiting for user
click
+          await LoadWebcams();
+          // iOS Autoplay Fix: Show tap-to-start screen instead of auto-
starting
+          ShowTapToStart();
```

Add the tap gate HTML and handler:

```
<!-- Add this after the advanced-loader div -->
<div id="tapToStartDiv" class="ctaDiv" style="display: none; background:
rgba(0,0,0,0.9);">
  <div style="text-align:center; color:white; font-family: -apple-system, sans-
serif;">
    <div style="font-size:48px; margin-bottom:20px;">📷</div>
    <p style="font-size:18px; font-weight:600; margin-bottom:8px;">AR
Experience Ready</p>
    <p style="font-size:14px; opacity:0.7; margin-bottom:30px;">Point your
camera at the image to begin</p>
    <button id="tapStartBtn" onclick="OnUserTapStart()" style="
padding: 16px 48px;
font-size: 16px;
font-weight: 700;
```

```

        background: linear-gradient(135deg, #00d2ff, #3a7bd5);
        color: white;
        border: none;
        border-radius: 50px;
        cursor: pointer;
        letter-spacing: 2px;
        text-transform: uppercase;
        ">TAP TO START</button>
    </div>
</div>

```

```

function ShowTapToStart() {
    document.getElementById('tapToStartDiv').style.display = 'flex';
}

function OnUserTapStart() {
    // === iOS AUTOPLAY UNLOCK ===
    // Creating and playing a silent audio context inside a user gesture
    // permanently unlocks media playback for this browsing session.
    try {
        var AudioContext = window.AudioContext || window.webkitAudioContext;
        var ctx = new AudioContext();
        var oscillator = ctx.createOscillator();
        oscillator.frequency.value = 0; // Silent
        oscillator.connect(ctx.destination);
        oscillator.start(0);
        oscillator.stop(0.001);

        // Also unlock <video> elements by playing a blank data URI
        var silentVideo = document.createElement('video');
        silentVideo.src =
'data:video/mp4;base64,AAAAIGZ0eXBpc29tAAACAGlzb21pc28yYXZjMW1wNDEAAAAIZnJlZQAAA..';
        silentVideo.muted = false;
        silentVideo.play().catch(function(){});
    } catch(e) {
        console.warn('[iOS Unlock] AudioContext unlock failed:', e);
    }

    document.getElementById('tapToStartDiv').style.display = 'none';
    StartAR();
}

```

[!NOTE] The `AudioContext` trick is the most reliable way to unlock iOS media. Once an `AudioContext` is created inside a `touchend/click` handler, **all subsequent** `.play()` calls are allowed — even from async callbacks like your AR tracker's `OnEnable()`.

Step 2: Mute-First Fallback in C# (Belt & Suspenders)

Even with the tap gate, add a safety net in `CDNARVideoController.cs` so the video at least plays muted on iOS if something goes wrong:

```
// In OnEnable(), before calling Play():
#if UNITY_WEBGL && !UNITY_EDITOR
    // iOS fallback: attempt muted playback first, then unmute
    videoPlayer.SetDirectAudioMute(0, true);
    videoPlayer.Play();
    StartCoroutine(TryUnmuteAfterDelay());
#else
    videoPlayer.Play();
#endif
```

```
private System.Collections.IEnumerator TryUnmuteAfterDelay()
{
    yield return new WaitForSeconds(0.5f);
    if (videoPlayer.isPlaying)
    {
        videoPlayer.SetDirectAudioMute(0, false);
    }
}
```

Fix B: Strip Audio from AR Videos at the CDN Level

If your AR videos are **purely visual** (e.g., a 3D model rotation, an animated logo), the absolute simplest fix is to strip the audio track from the `.mp4` before uploading:

```
# FFmpeg: Remove audio track entirely
ffmpeg -i input.mp4 -an -c:v copy output_noaudio.mp4
```

A `.mp4` with no audio track + the `playsinline` attribute = **iOS autoplay works with zero code changes**.

[!TIP] For your SaaS platform, add this as an automated post-processing step in your Node.js compiler microservice. When a client uploads a video, auto-generate both an audio and no-audio version.

Decision Matrix

Scenario	Fix A (Tap Gate)	Fix B (Strip Audio)
Videos with important audio	☑ Required	✗ Loses audio
Videos that are visual-only	☑ Works	☑ Simplest
Client-uploaded unknown content	☑ Universal	⚠ Risky if audio matters

Scenario	Fix A (Tap Gate)	Fix B (Strip Audio)
UX impact	One extra tap	Zero extra taps
Implementation effort	~2 hours	~30 min + FFmpeg pipeline

Recommendation

Use both:

- 1. **Fix A** as the universal safety net in your WebGL template — one tap unlocks everything
- 2. **Fix B** as an optimization in your SaaS upload pipeline — so visual-only AR experiences get zero-friction playback

This combination is exactly what platforms like 8thWall and Zappar use in production.